Game Spot Reservations: Service-Oriented Architecture with Two Providers and a Web Consumer

Cristian Farid Amado Peña and Cristian Santiago Silva Muñoz

Department of Systems Engineering, Universidad Santo Tomás, Villavicencio

Service-Oriented Development (Desarrollo Orientado a Servicios)

Diana Velásquez

November 18, 2026

Abstract

Game Spot is a video game lounge in Villavicencio, Colombia, that rents consoles by the hour. This document presents the service-oriented version of its booking system, built as three independent Spring Boot applications that communicate over HTTP with JSON. Provider 1 (port 8091) exposes the catalog of stations, hourly rates, and opening hours. Provider 2 (port 8092) manages bookings in 30-minute blocks and issues JSON Web Tokens. The consumer application (port 8080) serves a web page written in HTML, CSS, and JavaScript with the Fetch API, and its backend calls both providers with Spring RestClient to coordinate two integrated operations: the daily availability of a station and the creation of a booking. Each provider persists its data in its own H2 file database defined with SQL scripts and accessed through JPA repositories. The document describes the architecture, the API contract, the data transfer objects, the data model, the authentication design, and the tests: 24 automated tests across the three projects, a Postman collection of 40 requests with 43 assertions, and a manual end-to-end test that includes a stopped provider.

Keywords: service-oriented architecture, REST API, Spring Boot, RestClient, JSON Web Token, H2

Game Spot Reservations: Service-Oriented Architecture with Two Providers and a Web Consumer

Game Spot rents six stations: one Xbox 360, four Xbox Series S, and a VIP room with an Xbox Series X, at hourly rates of 5,000, 6,000, and 7,000 Colombian pesos. It opens from 2:00 p.m. to 10:00 p.m. on weekdays and from 3:00 p.m. to 9:00 p.m. on weekends. Customers used to book by phone, so the lounge had no reliable way to show free time slots, to prevent two people from booking the same station at the same time, or to keep a record of bookings. The team had already built a monolithic version of the system for another course; this project adapts the same business idea to the requirements of service-oriented development.

Splitting the system into services is justified because the two capabilities change for different reasons and at different times. The catalog changes when the owner buys a station, repairs one, or updates prices, and it is mostly read. Bookings change every time a customer reserves, pays, or cancels, and they carry the customer data and the authentication. Separating them lets each provider own its data and evolve independently, and the consumer shows how a client application combines both through their public contracts.

Objectives

The general objective is to build a web solution for booking Game Spot stations, organized as two provider applications and one consumer application that communicate through REST APIs. The specific objectives are to (a) expose the catalog of stations, rates, and opening hours as a provider with full CRUD operations, (b) expose bookings and authentication as a second provider, (c) build a consumer with a web interface that integrates both providers with RestClient, (d) persist each provider’s data in its own relational database with DDL, DML,

repositories, and ORM, (e) protect modifying operations with JWT, and (f) verify the solution with automated tests, Postman, and a test with a stopped provider.

Scope

The scope includes the three applications, the catalog CRUD, the booking life cycle (create, pay a simulated deposit, and cancel), the daily availability, the authentication of test users, and the web interface for customers and administrators. It excludes real online payments, e-mail notifications, user registration, deployment to a public server, and a production database. These features exist in the monolithic version of the project but are outside the scope of this delivery.

Users and Functionality

The solution has two kinds of users. A customer signs in, checks which 30-minute blocks of a station are free on a given day, books a block, pays the deposit, and cancels a booking. An administrator signs in and changes the catalog, for example by taking a station out of service or changing an hourly rate. Table 1 relates the functional requirements of the course guide to the place where the solution meets them.

Table 1

Functional Requirements and Where They Are Met

Code	Requirement	Implementation
RF-01	Solve the problem defined by the team	Online booking of Game Spot stations
RF-02	Provider 1 with a business capability	proveedor-catalogo: stations, rates, and opening hours
RF-03	Provider 2 with a different capability	proveedor-reservas: bookings and authentication
RF-04	Consumer connected to both providers	CatalogoCliente and ReservasCliente with RestClient
RF-05	At least two endpoints per provider	8 endpoints in Provider 1 and 7 in Provider 2
RF-06	GET, POST, and PUT, PATCH, or DELETE	All five methods are used
RF-07	Register information from the interface	Booking form and sign-in form

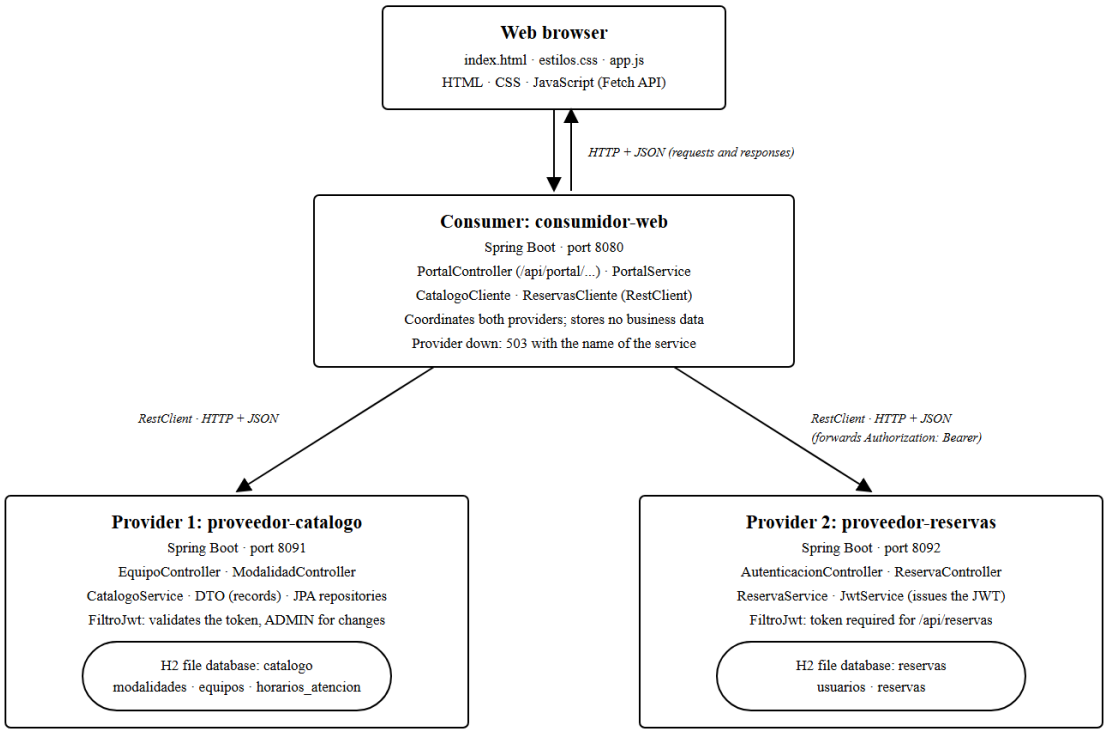
Code	Requirement	Implementation
RF-08	Query and show data in the interface	Stations, availability grid, and the customer's bookings
RF-09	Update, cancel, or manage information	Pay the deposit, cancel a booking, take a station out of service
RF-10	Integrate both providers in one operation	Daily availability and booking creation
RF-11	Validate input and show errors	Bean Validation, business rules, and messages in the interface
RF-12	Test a stopped provider	503 response and a working page (Figure 4)
RF-13	CRUD on project resources	Full CRUD of stations; logical cancellation of bookings
RF-14	JWT authentication and a protected operation	Login in Provider 2; tokens required for bookings and catalog changes
RF-15	Persistence that survives a restart	H2 file databases; verified by restarting the providers

Service-Oriented Architecture

Figure 1 shows the three applications. Each one is a separate Maven project with its own *pom.xml*, process, port, and configuration. The browser only talks to the consumer, and the consumer only talks to the providers through their HTTP interfaces; it never reads or writes the providers' databases. The provider URLs are properties of the consumer (*app.proveedores.catalogo.url* and *app.proveedores.reservas.url*) that can be changed with the environment variables CATALOGO_URL and RESERVAS_URL.

Figure 1

SOA Diagram of Game Spot



Note. Arrows show the direction of the requests; every response returns along the same path.

Table 2

Responsibilities and Data of Each Application

Application	Responsibility	Data it owns
proveedor-catalogo (8091)	Lists stations, rates, and opening hours; creates, updates, and deletes stations; changes rates. Validates the JWT and requires the ADMIN role for changes.	modalidades, equipos, and horarios_atencion in the H2 database catalogo
proveedor-reservas (8092)	Signs users in and issues the JWT; creates bookings with the block rules and overlap check; lists bookings and the occupancy of a day; pays and cancels.	usuarios and reservas in the H2 database reservas
consumidor-web (8080)	Serves the web page; calls both providers with RestClient; coordinates availability and booking; translates provider failures into clear messages.	None; it keeps no business data

API Contract

Tables 3 to 5 list the endpoints. All of them exchange JSON, and every error has the same body: a timestamp, the status code, a message in Spanish, and a list of details, for example one per invalid field.

Table 3

Provider 1 Endpoints (Catalog, Port 8091)

Method	Route	Token	Responses
GET	/api/equipos	No	200 list of stations
GET	/api/equipos/{id}	No	200 station; 404 not found
POST	/api/equipos	ADMIN	201 created; 400 invalid data; 404 unknown type; 409 duplicate name
PUT	/api/equipos/{id}	ADMIN	200 updated; 400; 404; 409
DELETE	/api/equipos/{id}	ADMIN	204 deleted; 404
GET	/api/modalidades	No	200 station types with their rate
PUT	/api/modalidades/{id}	ADMIN	200 updated; 400 rate outside 1 to 1,000,000; 404
GET	/api/horarios	No	200 opening hours of the seven days

Note. Routes that require a token answer 401 without a token or with an invalid one, and 403 with a customer token.

Table 4

Provider 2 Endpoints (Bookings, Port 8092)

Method	Route	Token	Responses
POST	/api/auth/login	No	200 token; 400 invalid data; 401 wrong credentials
POST	/api/reservas	Yes	201 booking pending payment; 400 invalid block; 409 block taken
GET	/api/reservas/mias	Yes	200 the user’s bookings, newest first
GET	/api/reservas/{id}	Yes	200 booking; 403 another user’s booking; 404
GET	/api/reservas/ocupacion?equipoId=&fecha=	No	200 occupied intervals of the day
PATCH	/api/reservas/{id}/pago	Yes	200 confirmed; 409 not pending
PATCH	/api/reservas/{id}/cancelacion	Yes	200 cancelled; 409 already cancelled

Table 5

Consumer Endpoints (Port 8080)

Method	Route	Providers used	Responses
GET	/api/portal/estado	Both	200 availability of each provider
GET	/api/portal/equipos	Catalog	200; 503 if the catalog is down
GET	/api/portal/equipos/{id}/disponibilidad	Both	200 blocks of the day with their state
POST	/api/portal/login	Bookings	200 token; 401
POST	/api/portal/reservas	Both	201; 400 outside opening hours; 401; 409; 503
GET	/api/portal/reservas/mias	Bookings	200
PATCH	/api/portal/reservas/{id}/pago and /cancelacion	Bookings	200; 409
PATCH	/api/portal/equipos/{id}/estado	Catalog	200; 401; 403

Example Exchange

The consumer creates a booking in Provider 2 with the following request, after reading the station name and rate from Provider 1:

```
POST http://localhost:8092/api/reservas
Authorization: Bearer eyJhbGciOiJIUzM4NCJ9...
{ "equipoId": 1, "nombreEquipo": "Xbox 360 #1", "tarifaHoraPesos": 5000,
  "inicio": "2026-11-10T21:00:00Z", "duracionMinutos": 60 }

201 Created
{ "id": 1, "equipoId": 1, "nombreEquipo": "Xbox 360 #1",
  "inicio": "2026-11-10T21:00:00Z", "fin": "2026-11-10T22:00:00Z",
  "precioTotalPesos": 5000, "adelantoPesos": 2500,
  "estado": "PENDIENTE_PAGO", "emailCliente": "laura@gamespot.demo" }
```

Data Transfer Objects

Every request and response body is a Java record, and input records carry Bean Validation annotations. Table 6 lists the main ones.

Table 6

Main Data Transfer Objects

DTO	Application	Fields and types	Use
EquipoRequest	Catalog	nombre: String (3 to 60); modalidadId: Long; estaDisponible: Boolean	Create or update a station
EquipoResponse	Catalog	id: Long; nombre: String; modalidadId: Long; modalidad: String; tarifaHoraPesos: int; estaDisponible, sePuedeReservar: boolean	Station returned by the API
TarifaRequest	Catalog	tarifaHoraPesos: Integer (1 to 1,000,000); estaActiva: Boolean	Change a rate
LoginRequest	Bookings	email: String (e-mail); password: String	Sign in
TokenResponse	Bookings	token, tipo, nombre, email, rol: String; expiraEn: Instant	Issued token
CrearReservaRequest	Bookings	equipoId: Long; nombreEquipo: String; tarifaHoraPesos: Integer; inicio: Instant; duracionMinutos: Integer (30 to 240)	Create a booking
ReservaResponse	Bookings	id, equipoId: Long; nombreEquipo: String; inicio, fin: Instant; precioTotalPesos, adelantoPesos: int; estado; emailCliente: String	Booking returned by the API
ReservaPortalRequest	Consumer	equipoId: Long; fecha: LocalDate; hora: LocalTime; duracionMinutos: Integer	Booking form of the web page
DisponibilidadResponse	Consumer	equipo; fecha: LocalDate; atiende: boolean; bloques: list of {inicio, fin, estaLibre, motivo}	Availability grid
ApiError	All	timestamp: Instant; status: int; mensaje: String; detalles: List<String>	Every error response

Application Design

Controller, Service, and Repository Layers

The three applications follow the same layered structure. Controllers, such as *EquipoController*, *ReservaController*, and *PortalController*, map routes and HTTP methods, validate the input with *@Valid*, and delegate to a service; they contain no business rules. Services, such as *CatalogoService*, *ReservaService*, and *PortalService*, hold the rules and run inside transactions. In the providers, services read and write data only through Spring Data JPA

repositories, which play the role of the Data Access Object pattern: *EquipoRepository*, for example, declares *findAllByOrderByModalidadTarifaHoraPesosAscNombreAsc* and *existsByNombreIgnoreCase*, and *ReservaRepository* declares a JPQL query that finds active bookings of a station that overlap an interval. A global exception handler in each application turns exceptions into *ApiError* responses with the right status code.

RestClient in the Consumer

The consumer defines one RestClient per provider in *ClientesConfig*, with the provider's base URL, a connection timeout of 2 seconds, and a read timeout of 5 seconds. *CatalogoCliente* and *ReservasCliente* wrap every call: they build the request, forward the *Authorization* header of the browser when the operation is protected, and convert the JSON response into a DTO. A shared helper translates failures. A connection error or a timeout becomes a 503 response that names the unavailable service, and an error response from a provider reaches the browser with the same status code and message.

Integrated Operations

The consumer coordinates both providers in two operations. To show the availability of a station on a day, it reads the station and the opening hours from Provider 1 and the occupied intervals from Provider 2, and it marks each 30-minute block as free, occupied, past, or out of service. To create a booking, it follows these steps: (a) the browser sends the station, date, start time, and duration to POST /api/portal/reservas with the token; (b) the consumer reads the station from Provider 1 and rejects it with 409 if it is out of service; (c) it reads the opening hours from Provider 1 and rejects the booking with 400 if it falls outside them; (d) it sends the station name, the current rate, and the start time converted to Bogotá time to Provider 2 with the same token; (e) Provider 2 validates the block, checks overlaps, computes the price and the 50% deposit, and

stores the booking; and (f) the consumer returns the booking to the browser, which refreshes the grid and the list of bookings.

Web Interface

The consumer serves *index.html*, *estilos.css*, and *app.js* as static resources. The script uses the Fetch API for every request to */api/portal*, keeps the token in the browser's session storage after sign-in, and adds it to protected requests. The page shows the status of both providers, the table of stations (with buttons for the administrator), the availability grid, the booking form, and the customer's bookings with buttons to pay the deposit and to cancel. Error messages from the backend are shown as they arrive, in Spanish.

Data Model and Persistence

DDL

Each provider defines its tables in *schema.sql*, which Spring Boot runs at start-up. The statements use *create table if not exists*, so an existing database keeps its data. Provider 1 has *modalidades* (identity key, unique name, hourly rate between 1 and 1,000,000, and an active flag), *equipos* (identity key, unique name, foreign key *modalidad_id* to *modalidades*, and an availability flag), and *horarios_atencion* (day of the week from 1 to 7 as the key, and opening and closing times with a check that closing is later). Provider 2 has *usuarios* (identity key, unique e-mail, BCrypt password hash, and a role restricted to CLIENTE or ADMIN) and *reservas* (identity key, foreign key *usuario_id*, station id, station name, start, end, price, deposit, state restricted to the three valid values, and checks that the end is after the start and the deposit does not exceed the price). The station id in *reservas* has no foreign key because stations live in another application with its own database.

DML and CRUD

Each provider loads its initial data with *data.sql*. Provider 1 inserts the real stations, types, and opening hours, and Provider 2 inserts three test users. Every insert runs only if the row does not exist, so restarting the application does not duplicate data or undo changes made through the API. Table 7 shows how each CRUD operation of the API corresponds to SQL.

Table 7

Correspondence Between CRUD, HTTP, and SQL

CRUD	HTTP endpoint	SQL executed through JPA
Create	POST /api/equipos; POST /api/reservas	INSERT INTO equipos ...; INSERT INTO reservas ...
Read	GET /api/equipos; GET /api/reservas/mias	SELECT ... FROM equipos JOIN modalidades ...; SELECT ... FROM reservas WHERE usuario ...
Update	PUT /api/equipos/{id}; PUT /api/modalidades/{id}; PATCH /api/reservas/{id}/pago	UPDATE equipos SET ...; UPDATE modalidades SET ...; UPDATE reservas SET estado = ...
Delete	DELETE /api/equipos/{id}	DELETE FROM equipos WHERE id = ?
Logical delete	PATCH /api/reservas/{id}/cancelacion	UPDATE reservas SET estado = ’CANCELADA’, cancelada_en = ...

Note. Bookings are cancelled instead of deleted because the lounge needs to keep the history of what was reserved and paid; a cancelled booking frees its block.

ORM

The providers use JPA with Hibernate to map entities to tables. The *Equipo* entity, for example, is annotated with *@Entity* and *@Table(name = "equipos")*; its *id* field uses *@GeneratedValue(strategy = IDENTITY)*; its *estaDisponible* field maps to the column *esta_disponible*; and its *modalidad* field uses *@ManyToOne* with *@JoinColumn(name = "modalidad_id")*, so loading a station also loads its type and rate. Enumerations such as *EstadoReserva* are stored as text with *@Enumerated(EnumType.STRING)*. Hibernate runs with

ddl-auto: validate: it does not create tables, it only checks at start-up that the entities match the tables created by *schema.sql*.

Persistence Mechanism

Each provider uses an H2 database stored in a file (*./data/catalogo* and *./data/reservas*), as allowed by the course guide, so the data survives a restart and no database server is needed. The team verified this by creating and paying a booking, stopping both providers, starting them again, and finding the booking still confirmed. The automated tests use in-memory H2 databases created from the same scripts.

Authentication and Authorization With JWT

Provider 2 authenticates users. POST /api/auth/login compares the password with its BCrypt hash and returns a JSON Web Token (Jones et al., 2015) signed with HMAC-SHA. The token carries the e-mail as subject, the role and name as claims, and an expiration of two hours. The browser keeps the token and sends it as *Authorization: Bearer* in every protected request; the consumer forwards it unchanged to the provider that owns the operation.

Both providers validate tokens with a servlet filter and the same secret, read from the environment variable JWT_SECRETO and never written in the repository. In Provider 2, every route under */api/reservas* requires a valid token except the public occupancy query, and the service also checks that a customer only reads, pays, or cancels their own bookings (403 otherwise). In Provider 1, queries are public, and creating, updating, or deleting requires a token whose role is ADMIN. A request without a token or with an invalid or expired one receives 401, and a valid token without the required role receives 403. The consumer does not validate tokens itself; it relies on the providers, which are the owners of the protected resources.

Testing

Automated Tests

Each project has its own automated tests, run with *./mvnw test*. Table 8 summarizes them. The provider tests start the application with an in-memory H2 database and call the endpoints through MockMvc with real tokens. The consumer tests replace the providers with test doubles to check the coordination logic, and a separate class uses a mock HTTP server and a closed port to check the RestClient calls, the forwarded token, the translation of provider errors, and the behavior with a stopped provider.

Table 8

Automated Tests by Project

Project	Test class and cases	Tests	Result
proveedor-catalogo	CatalogoTests, CP-36a to CP-36g: seed data, query and 404, full CRUD, validation and duplicates, 401 and 403, rate change, opening hours	7	0 failures
proveedor-reservas	ReservasTests, CP-37a to CP-37h: login, price and deposit, invalid blocks, overlaps, own bookings and public occupancy, payment, logical cancellation and 403, 401	8	0 failures
consumidor-web	PortalTests and ClientesHttpTests, CP-38a to CP-38i: integrated availability and booking, business validation, stopped provider, web page, RestClient routes and errors	9	0 failures

Postman

The collection *soa/postman/GameSpot-SOA.postman_collection.json* has 40 requests in five folders: authentication, Provider 1, Provider 2, consumer, and stopped provider. It stores the tokens and created identifiers in collection variables and computes tomorrow’s date in Bogotá time before the booking requests, so it can be run repeatedly. Every request has a test that checks its status code, and some also check values, such as the 50% deposit. The team ran the whole collection with Newman against the three applications on November 10, 2026: the first four

folders passed 38 requests and 40 assertions, and, after stopping Provider 1, the last folder passed its 2 requests and 3 assertions, with no failures.

End-to-End Test

The team started the three applications and used the web page as a customer. Figure 2 shows the signed-in customer and the stations read from Provider 1. Figure 3 shows a booking at 4:00 p.m. created through the integrated operation: the grid marks the two blocks as occupied and the booking appears in the customer's list. The deposit was then paid and the booking changed to confirmed.

Figure 2

Signed-In Customer and Stations From Provider 1

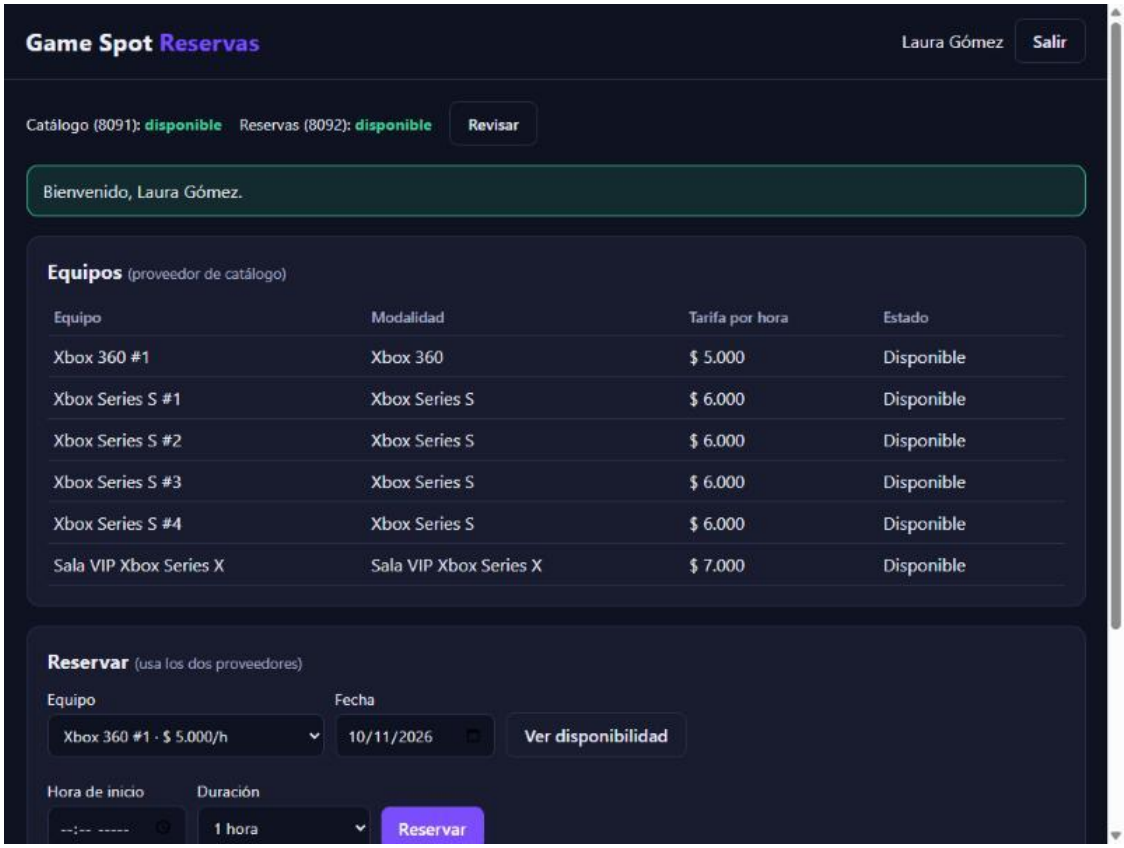
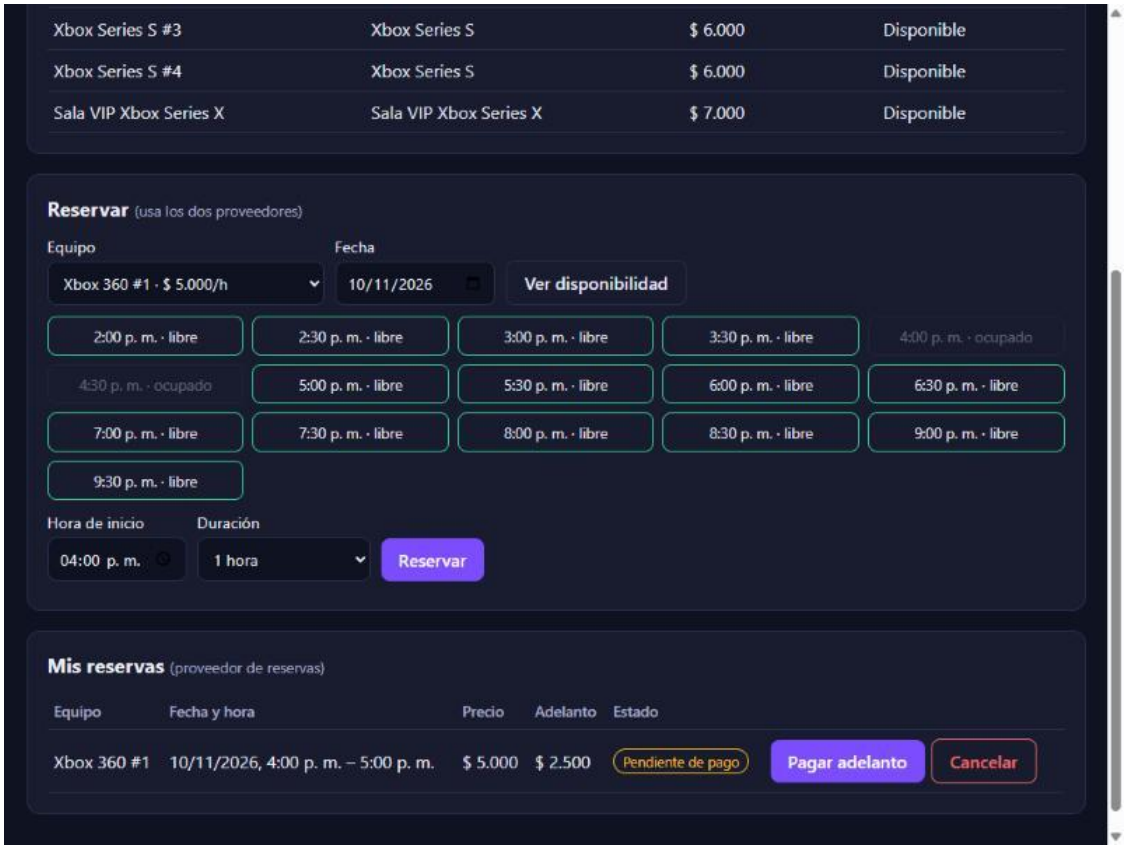


Figure 3

Booking Created With Both Providers

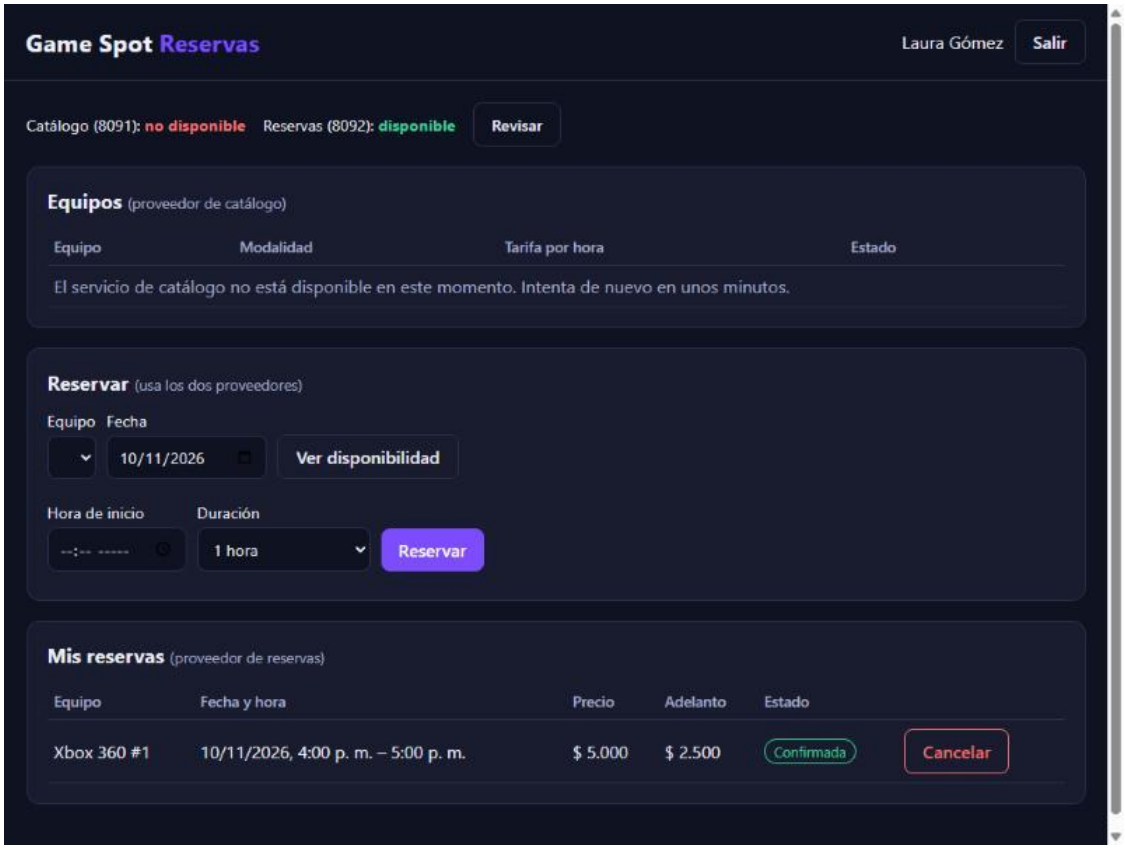


Stopped Provider

With the three applications running, the team stopped Provider 1. As Figure 4 shows, the page kept working: the status bar marks the catalog as unavailable, the table of stations shows the message returned by the consumer, and the customer's bookings, which come from Provider 2, are still listed and can be cancelled. The consumer answered GET /api/portal/equipos with status 503 and the message that the catalog service is not available, instead of an unhandled error.

Figure 4

Web Page With the Catalog Provider Stopped



Execution Guide

The applications need JDK 21. Docker and an external database are not needed. Before starting the providers, define the environment variable JWT_SECRETO with the same value of at least 32 characters in both of them, for example in the run configurations of IntelliJ IDEA. Then start, in this order, *soa/proveedor-catalogo*, *soa/proveedor-reservas*, and *soa/consumidor-web* with *./mvnw spring-boot:run* (or *.\mvnw.cmd* in PowerShell), and open http://localhost:8080. Table 9 lists the configuration of each application.

Table 9

Configuration of Each Application

Application	Port	Main settings
proveedor-catalogo	8091	JWT_SECRETO; H2 file at ./data/catalogo
proveedor-reservas	8092	JWT_SECRETO; H2 file at ./data/reservas; tokens valid for 2 hours
consumidor-web	8080	CATALOGO_URL (default http://localhost:8091); RESERVAS_URL (default http://localhost:8092); timeouts of 2 and 5 seconds

Note. Test accounts: admin@gamespot.demo with password AdminDemo123, and laura@gamespot.demo or andres@gamespot.demo with password ClienteDemo123. They are demonstration data, not real credentials.

Conclusions, Difficulties, and Lessons Learned

The solution meets the minimum architecture of the course: two providers with different capabilities, a consumer that integrates them with RestClient, a web interface with Fetch, persistence with DDL, DML, repositories, and ORM, and JWT protecting the modifying operations. The integrated booking shows the main benefit of the design: the consumer composes the catalog and the bookings without either provider knowing about the other, and a failure in one provider affects only the features that depend on it.

The main difficulty was that the team started from a monolithic system with a single database that enforced the booking rules with constraints and triggers. Splitting it meant deciding which application owns each piece of data and moving some rules to the services, such as the overlap check, which the monolith enforced with a PostgreSQL exclusion constraint that H2 does not offer. A second difficulty was security across applications: both providers had to validate the same tokens without calling each other, which the team solved with a shared secret provided through an environment variable. A third one was making failures visible to the user, which required timeouts in the RestClient and a uniform error body in all three applications.

The team learned that a service contract has to be fixed before the consumer is built, that a consumer must treat every remote call as something that can fail, and that copying the data a service needs, such as the station name and price in a booking, keeps the services independent at the cost of some duplication.

References

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

H2 Database Engine. (n.d.). *H2 database engine*. Retrieved November 10, 2026, from https://www.h2database.com/html/main.html

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Postman. (n.d.). *Postman learning center*. Retrieved November 10, 2026, from https://learning.postman.com/docs/

Spring. (n.d.). *REST clients*. Spring Framework reference documentation. Retrieved November 10, 2026, from https://docs.spring.io/spring-framework/reference/integration/rest-clients.html

Appendix

Statement on the Use of Artificial Intelligence

The team used an AI coding assistant (Claude, by Anthropic) to support the writing of code, tests, the Postman collection, and this document. The team defined the business idea, the division into services, and the design decisions, reviewed every pull request, and verified the results reported here by running the tests, the Postman collection, and the applications.